

Reviewer Pack — Minimal Rank of Quasi-Symmetric Functions over Tropical Semi...

Entry d0aa5ddc855c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 01:52:54 UTC

Minimal Rank of Quasi-Symmetric Functions over Tropical Semirings vs BP_ReadTwice Complexity

Entry ID: d0aa5ddc855c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 01:52:54 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Symmetry Theory **Field B** (complexity object): Complexity Theory: BP_ReadTwice Complexity

Statement:

[‘For a given read-twice branching program P with n variables, the minimal rank of its associated quasi-symmetric function $f(P)$ over the tropical semiring is $O(\log(n))$ but for the trivial IP_2 branching program, the minimal rank of $f(IP_2)$ is $\tilde{O}(n)$.’, ‘Equivalently, there exists a constructive mapping from P to $f(P)$ such that BP_ReadTwice complexity of P is upper-bounded by the logarithm of the minimal rank of $f(P)$.’]

Rationale (proposer’s reasoning):

[‘Quasi-symmetric functions are algebraic objects that can capture non-commutative properties in a way that is similar to symmetric functions. By connecting quasi-symmetric functions with tropical semirings, we may uncover new ways to measure complexity-theoretic objects like branching programs.’, ‘The conjecture suggests that the structure of tropical quasisymmetric functions could be used to distinguish between complex and simple read-twice branching programs, providing a potential tool for proving lower bounds on BP_ReadTwice complexity.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fc5c4ac58cd3ef8c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of 30 randomly generated read-twice branching programs, the BP_ReadTwice complexity is within a factor of 2 of $\log(\text{minimal rank}(f(P)))$ over the tropical semiring.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -quasi-symmetric functions AND tropical semirings AND BP_ReadTwice complexity-minimal rank quasi-symmetric functions tropical semiring IP_2 branching program
-constructive mapping BP_ReadTwice complexity minimal rank quasi-symmetric function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tropical_add(x, y):
        return max(x, y)

    def tropical_multiply(x, y):
        if x == float('-inf') or y == float('-inf'):
            return float('-inf')
        return x + y

    def tropical_negate(x):
        return -x

    def tropical_zero():
        return float('-inf')

    def tropical_one():
        return 0

    def tropical_is_zero(x):
        return x == float('-inf')

    def tropical_rank(f):
        n = len(f)
        if n == 1:
            return 1
```

```

    rank = 1
    for i in range(1, n):
        if not any(tropical_is_zero(f[i][j]) for j in range(i)):
            rank += 1
    return rank

def BP_ReadTwice_complexity(P):
    n = len(P)
    t_star = [0] * (n + 1)
    t_star[0] = 1
    for i in range(n):
        t_star[i+1] = sum(tropical_multiply(t_star[j], P[i][j]) for j in range(i+1))
    return max(t_star)

def quasi_symmetric_function(P):
    n = len(P)
    f = [[tropical_zero() for _ in range(n)] for _ in range(n)]
    for i in range(n):
        f[0][i] = P[i][0]
    for i in range(1, n):
        for j in range(i+1):
            f[j][i] = tropical_add(f[j-1][i], tropical_multiply(P[i][j], f[j][i-1]))
    return f

def generate_read_twice_branching_program(n):
    P = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1):
            P[i][j] = random.choice([tropical_zero(), tropical_one()])
    return P

n = 40
P = generate_read_twice_branching_program(n)
f = quasi_symmetric_function(P)
rank = tropical_rank(f)
bp_complexity = BP_ReadTwice_complexity(P)

if rank == 1:
    counterexample = "trivial_IP_2"
    conjecture_holds = False
else:
    ratio = bp_complexity / math.log(rank, 2)
    conjecture_holds = abs(ratio - 1) <= 0.5
    counterexample = ""

return {
    "metric_name": "BP_ReadTwice_complexity vs tropical rank",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

total_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={total_ratio} std=0 support_fraction={support_fraction}")
elif any(r["counterexample"] == "trivial_IP_2" for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if r["counterexample"] == "trivial_IP_2")
    print(f"RESULT: FALSIFIED counterexample=\"trivial_IP_2\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22dbc206.py", line 104, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22dbc206.py", line 92, in run_trial
    "metric_value": ratio,
                   ~~~~~
UnboundLocalError: cannot access local variable 'ratio' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12060
2	preregistration	ollama_remote	glm4:latest	0	0	5776
3	novelty	ollama_remote	glm4:latest	0	0	4749
4	novelty	ollama_remote	glm4:latest	0	0	9105
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43877
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8498
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9244
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10667
9	judge	ollama_remote	glm4:latest	0	0	44449

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148425 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d0aa5ddc855c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d0aa5ddc855c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d0aa5ddc855c.tar.gz` (if generated)